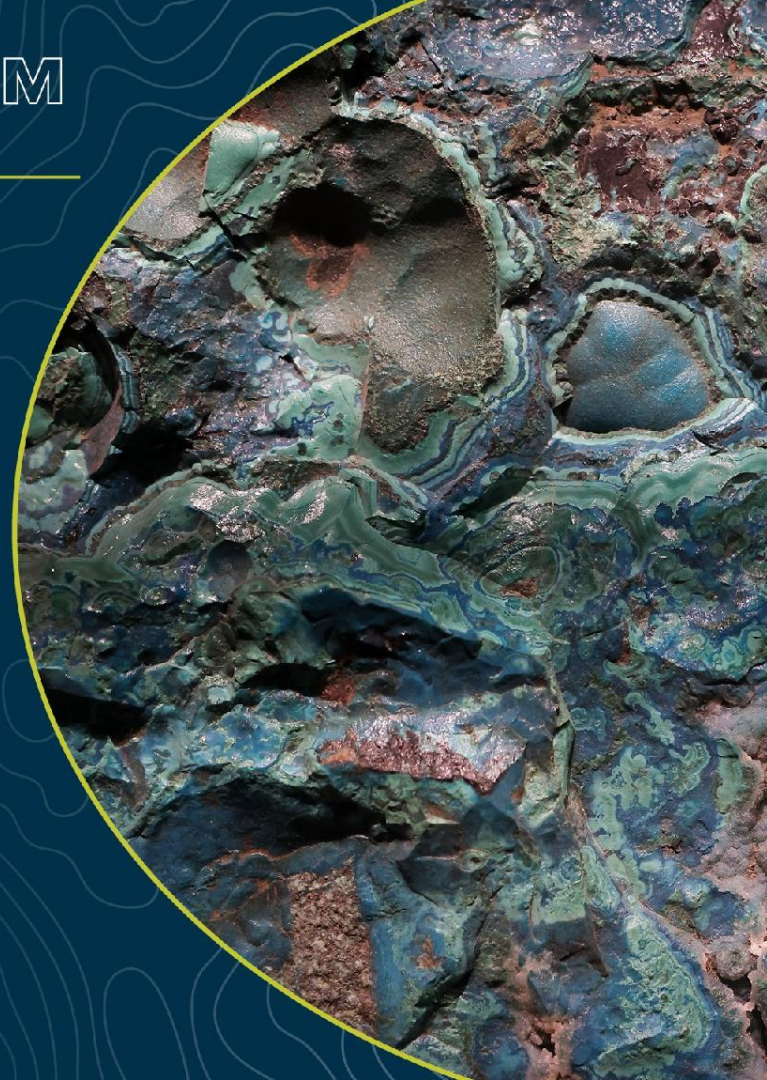
SEG CRITICAL MINERALS FORUM
27–29 October 2026 • Vancouver, British Columbia, Canada

On the redesign of SimPEG's inversion framework

Santiago Soler



simpeg

Simulation and parameter estimation in geophysics

common framework for simulations & inversions

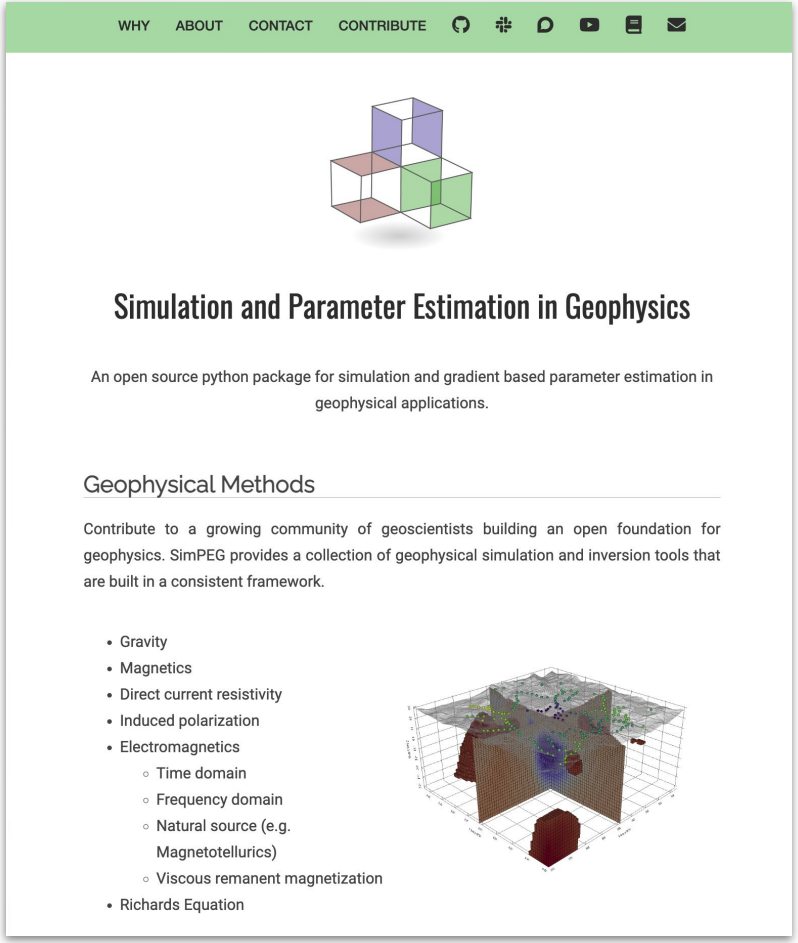
accelerate research: build upon others work

facilitate reproducibility of results

build & deploy in python

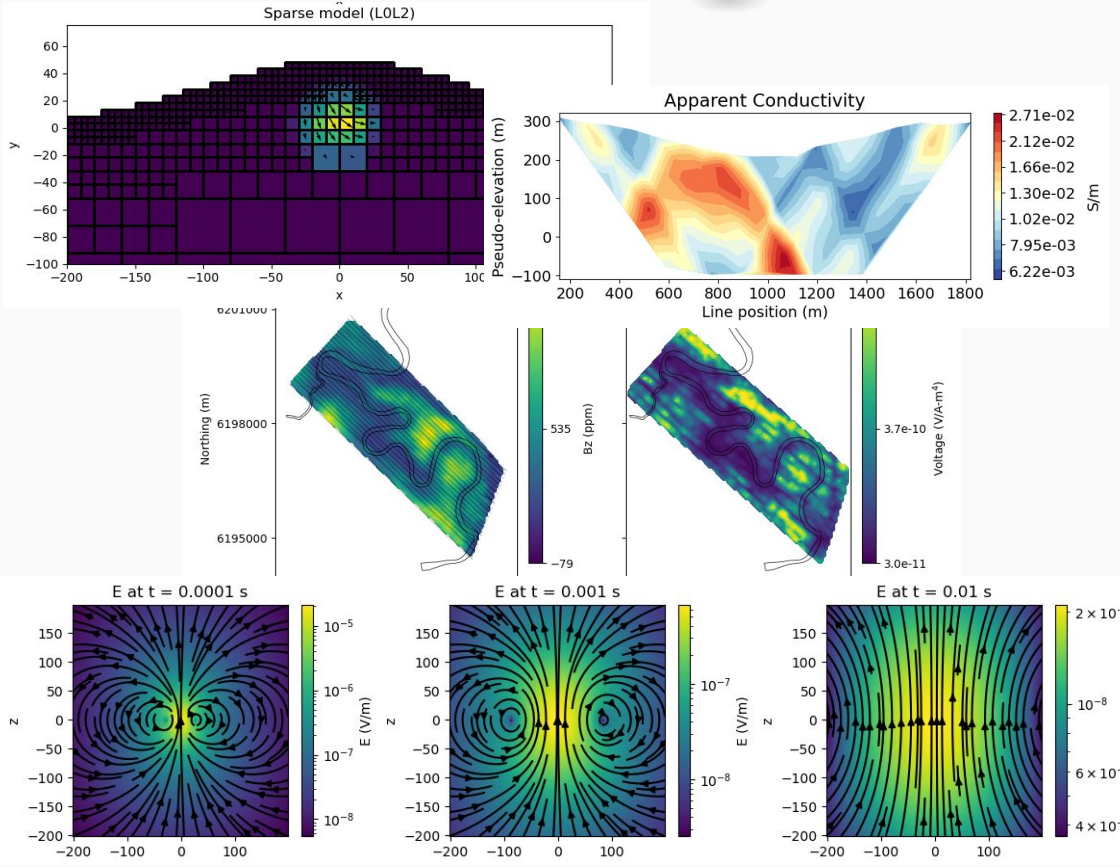
open-source

https://simpeg.xyz



Simulation of physics

- Gravity and magnetic
- DC-IP
- Frequency Domain EM
- Time Domain EM
- Natural Source EM
  - MT
  - Airborne NSEM systems



Deterministic inversions

Solves deterministic inversions of the type:

$$\min_{\mathbf{m}} \phi(\mathbf{m}) = \underbrace{\phi_{\mathrm{d}}(\mathbf{m})}_{\text{Data misfit}} + \underbrace{\beta\phi_{\mathrm{m}}(\mathbf{m})}_{\text{Model norm}}$$

Different type of inversions:

- Linear and non-linear inversions
- L2 inversions
- Sparse (Lp) inversions
- Petrophysical and geologically guided inversions (PGI)
- Parametric inversions

Issues with current framework

Dated codebase

Hard-coded assumptions

Non-trivial to replace or extend some pieces

Loaded interfaces: hinder interoperability

Redesign the inversion framework

Redesign the inversion framework

Goals

- Redesign the inversion framework
- More modular
- More extensible
- Minimal and clear interfaces
- Reduced complexity
- Reduce hard-coded assumptions

Broadly

- Accelerate research:
  - Less time coding.
  - More time researching.
- Aid learning: easier to study.
- Extend it: easily plug your code.

New inversion framework

- Source code:

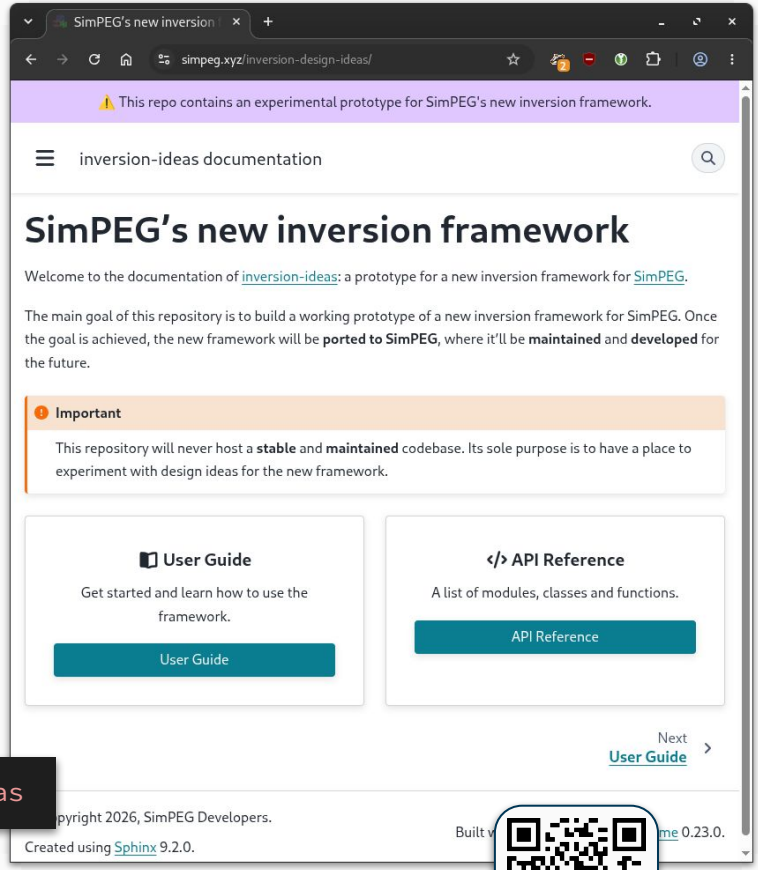
  simpeg/inversion-design-ideas

- Docs:

  https://simpeg.xyz/inversion-design-ideas

- Install it with:

```
pip install git+https://github.com/simpeg/inversion-design-ideas
```



New design

Fundamentals

Solve inversion problems of the type:

$$\min_{\mathbf{m}} \phi(\mathbf{m})$$

- Where $\phi(\mathbf{m})$ is any arbitrary **objective function**.
- Through any **minimizer** of choice.

Not necessarily of the form:
$\phi(\mathbf{m}) = \phi_{\mathrm{d}}(\mathbf{m}) + \beta\phi_{\mathrm{m}}(\mathbf{m})$

Objective functions

Objective functions represented by the `Objective` class.

- `Objective.__call__(model)`

  Evaluates the function: $\phi(\mathbf{m})$

- `Objective.gradient(model)`

  Gradient of the function: $\nabla\phi(\mathbf{m})$

- `Objective.hessian(model)`

  Hessian matrix of the function:

$$\nabla^2\phi(\mathbf{m}) = \begin{bmatrix} \partial^2{}_{11}\phi & \dots & \partial^2{}_{1M}\phi \\ \vdots & \ddots & \vdots \\ \partial^2{}_{M1}\phi & \dots & \partial^2{}_{MM}\phi \end{bmatrix}$$

```python
class Objective(ABC):

    def __call__(self, model) -> float:
        ...

    def gradient(self, model) -> NDArray:
        ...

    def hessian(
        self, model
    ) -> NDArray | SparseArray | LinearOperator:
        ...
```

Hessian can be a **dense** or **sparse** array, or a SciPy **LinearOperator**

Objective functions

We can define any linear combination of `Objective` objects.

```
import inversion_ideas as ii

data_misfit = ii.DataMisfit(...)
smallness = ii.Smallness(...)
flatness = ii.Flatness(..)

phi = data_misfit + 1e-3 * (smallness + 1e2 * flatness)
print(phi)
```

```
ϕd(m) + 0.001 [ϕs(m) + 100.0 ϕx(m)]
```

The resulting `phi` is another `Objective` object.

Minimizers

Function that can minimize any objective function:

Conjugate gradient: only for linear inversion problems

```
inverted_model = ii.minimize.conjugate_gradient(phi, intial_model)
```

Classes we can iterate over:

```
minimizer = ii.minimize.GaussNewtonConjugateGradient()
for model in minimizer(phi, initial_model):
    # Can insert code here:
    # print information, plot, etc.
    ...

# Last model is the inverted model
inverted_model = model
```

Gauss-Newton CG: for non-linear inversion problems

Physics in the mix: `Simulation`

The `DataMisfit` measures how well the predicted data fits the observed data.

$$\phi_d(\mathbf{m}) = \sum_{i=1}^{N} \frac{\left| f_i(\mathbf{m}) - d_i^{\text{obs}} \right|^2}{\epsilon_i^2} = \left\| \mathbf{W} \left[\mathbf{f}(\mathbf{m}) - \mathbf{d}^{\text{obs}} \right] \right\|^2$$

Data uncertainties

Forward modelling function

Observed data

```
data_misfit = ii.DataMisfit(data, uncertainties, simulation)
```

Data array

Array of data uncertainties

A `Simulation` object

Physics in the mix: `Simulation`

The forward modelling functions are represented by `Simulation` objects.

- `Simulation.__call__(model)`

  Predicts the data for a given model: $\mathbf{f}(\mathbf{m})$

- `Simulation.jacobian(model)`

  Jacobian matrix of $\mathbf{f}(\mathbf{m})$:

$$\mathbf{J}_\mathbf{f}(\mathbf{m}) = \begin{bmatrix} \frac{\partial f_1(\mathbf{m})}{\partial m_1} & \cdots & \frac{\partial f_1(\mathbf{m})}{\partial m_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_N(\mathbf{m})}{\partial m_1} & \cdots & \frac{\partial f_N(\mathbf{m})}{\partial m_M} \end{bmatrix}$$

```python
class Simulation(ABC):

    def __call__(self, model) -> float:
        ...

    def jacobian(
        self, model
    ) -> NDArray | SparseArray | LinearOperator:
        ...
```

Jacobian can be a **dense** or **sparse** array, or a SciPy **LinearOperator**

A primer on SciPy's `LinearOperator`

SciPy's `LinearOperator` are objects that can **represent a matrix** without storing it in memory.

Can compute dot product with vectors:

```
matrix = MyMatrix(...)
matrix @ vector
```

And with its transpose:

```
matrix.T @ vector
```

Need to implement methods for those dot products.

```python
class MyMatrix(LinearOperator):

    def matvec(self, vector):
        """Dot product with vector"""
        ...
        return result

    def rmatvec(self, vector):
        """Dot product of transpose with vector"""
        ...
        return result
```

A **LinearOperator** can represent a large matrix we cannot store in memory.

- `Simulation.jacobian()` — For large problems
- `DataMisfit.hessian()` — For most cases!

Derivatives: `Simulation` and `DataMisfit`

`Simulation`

The Jacobian can be a `LinearOperator`

```
jac = simulation.jacobian(model)
jac
```

```
<15x100 LinearOperator with dtype=float64>
```

We can compute dot products with it:

```
jac @ vector
```

```
jac.T @ vector
```

`DataMisfit`

The gradient can use that Jacobian:

```
def gradient(self, model):
    jac = self.simulation.jacobian(model)
    w = self.weights_matrix
    residual = self.residual(model)
    return 2 * jac.T @ (w.T @ w @ residual)
```

The Hessian will be another `LinearOperator`

```
def hessian(self, model):
    ...
    return 2 * jac.T @ w.T @ w @ jac
```

```
data_misfit.hessian(model)
```

```
<100x100 _ProductLinearOperator with dtype=float64>
```

Example:
Gravity inversion

Example: gravity inversion

```python
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)
# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

Example: gravity inversion

```
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)
# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

Example: gravity inversion

```
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)
# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

Example: gravity inversion

```
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)
# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

Example: gravity inversion

```python
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)

# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

Example: gravity inversion

```
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)
# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

Example: gravity inversion

```python
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)

# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

Example: gravity inversion

```python
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)
# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

```python
# Create objective function
beta = 1e5
phi = data_misfit + beta * model_norm

# Choose conjugate gradient as minimizer
minimizer = ii.minimize.conjugate_gradient

# Run inversion with conjugate gradient
initial_model = ...
inverted_model = minimizer(phi, initial_model)
```

Example: gravity inversion

```python
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)
# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

```python
# Create objective function
beta = 1e5
phi = data_misfit + beta * model_norm

# Choose conjugate gradient as minimizer
minimizer = ii.minimize.conjugate_gradient

# Run inversion with conjugate gradient
initial_model = ...
inverted_model = minimizer(phi, initial_model)
```

Example: gravity inversion

```python
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)

# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

```python
# Create objective function
beta = 1e5
phi = data_misfit + beta * model_norm

# Choose conjugate gradient as minimizer
minimizer = ii.minimize.conjugate_gradient

# Run inversion with conjugate gradient
initial_model = ...
inverted_model = minimizer(phi, initial_model)
```

Example: gravity inversion

```python
from simpeg.potential_fields import gravity

import inversion_ideas as ii

# Build SimPEG gravity simulation as usual
simpeg_simulation = gravity.Simulation3DIntegral(...)

# Wrap SimPEG's simulation
simulation = ii.WrappedSimulation(simpeg_simulation)

# Define data misfit
data_misfit = ii.DataMisfit(
    data, uncertainties, simulation
)

# Define regularization
smallness = ii.Smallness(mesh, cell_weights=weights)
flatness_x, flatness_y, flatness_z = (
    ii.Flatness(mesh, direction=d, cell_weights=weights)
    for d in ("x", "y", "z")
)
model_norm = (
    smallness
    + 6.0 * flatness_x
    + 6.0 * flatness_y
    + 12.0 * flatness_z
)
```

```python
# Create objective function
beta = 1e5
phi = data_misfit + beta * model_norm

# Choose conjugate gradient as minimizer
minimizer = ii.minimize.conjugate_gradient

# Run inversion with conjugate gradient
initial_model = ...
inverted_model = minimizer(phi, initial_model)
```

Inversions with multiple minimizations

Inversion with beta-cooling schedule

1. Define objective function with high value for β:

$$\phi(\mathbf{m}) = \phi_{\mathrm{d}}(\mathbf{m}) + \beta_0 \phi_{\mathrm{m}}(\mathbf{m})$$

2. Minimize objective function:

$$\mathbf{m}_{k+1} = \min_{\mathbf{m}} \phi(\mathbf{m})$$

3. Stop the inversion if a certain condition is met.

4. Cool down β by a constant factor:

$$\beta_{k+1} = \frac{\beta_k}{2}$$

Inversion with beta-cooling schedule

1. Define objective function with high value for β:

$$\phi(\mathbf{m}) = \phi_\mathrm{d}(\mathbf{m}) + \beta_0 \phi_\mathrm{m}(\mathbf{m})$$

2. Minimize objective function:

$$\mathbf{m}_{k+1} = \min_{\mathbf{m}} \phi(\mathbf{m})$$

3. Stop the inversion if a certain condition is met.

4. Cool down β by a constant factor:

$$\beta_{k+1} = \frac{\beta_k}{2}$$

```python
# Define objective function with dynamic beta
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Run inversion with beta-cooling schedule
model = initial_model.copy()
while True:
    # Perform a minimization step
    model = minimizer(phi, model)

    # Finish the inversion if reached the target
    if data_misfit.chi_factor(model) <= 1.0:
        break

    # Cool down beta by a factor of 2.0
    beta.update()

# Last model is the inverted one
inverted_model = model.copy()
```

Inversion with beta-cooling schedule

1. Define objective function with high value for β:

$$\phi(\mathbf{m}) = \phi_\mathrm{d}(\mathbf{m}) + \beta_0 \phi_\mathrm{m}(\mathbf{m})$$

2. Minimize objective function:

$$\mathbf{m}_{k+1} = \min_{\mathbf{m}} \phi(\mathbf{m})$$

3. Stop the inversion if a certain condition is met.
4. Cool down β by a constant factor:

$$\beta_{k+1} = \frac{\beta_k}{2}$$

```python
# Define objective function with dynamic beta
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Run inversion with beta-cooling schedule
model = initial_model.copy()
while True:
    # Perform a minimization step
    model = minimizer(phi, model)

    # Finish the inversion if reached the target
    if data_misfit.chi_factor(model) <= 1.0:
        break

    # Cool down beta by a factor of 2.0
    beta.update()

# Last model is the inverted one
inverted_model = model.copy()
```

Inversion with beta-cooling schedule

1. Define objective function with high value for β:

$$\phi(\mathbf{m}) = \phi_\mathrm{d}(\mathbf{m}) + \beta_0 \phi_\mathrm{m}(\mathbf{m})$$

2. Minimize objective function:

$$\mathbf{m}_{k+1} = \min_{\mathbf{m}} \phi(\mathbf{m})$$

3. Stop the inversion if a certain condition is met.

4. Cool down β by a constant factor:

$$\beta_{k+1} = \frac{\beta_k}{2}$$

```python
# Define objective function with dynamic beta
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Run inversion with beta-cooling schedule
model = initial_model.copy()
while True:
    # Perform a minimization step
    model = minimizer(phi, model)

    # Finish the inversion if reached the target
    if data_misfit.chi_factor(model) <= 1.0:
        break

    # Cool down beta by a factor of 2.0
    beta.update()

# Last model is the inverted one
inverted_model = model.copy()
```

Inversion with beta-cooling schedule

1. Define objective function with high value for β:

$$\phi(\mathbf{m}) = \phi_\mathrm{d}(\mathbf{m}) + \beta_0 \phi_\mathrm{m}(\mathbf{m})$$

2. Minimize objective function:

$$\mathbf{m}_{k+1} = \min_{\mathbf{m}} \phi(\mathbf{m})$$

3. Stop the inversion if a certain condition is met.

4. Cool down β by a constant factor:

$$\beta_{k+1} = \frac{\beta_k}{2}$$

```python
# Define objective function with dynamic beta
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Run inversion with beta-cooling schedule
model = initial_model.copy()
while True:
    # Perform a minimization step
    model = minimizer(phi, model)

    # Finish the inversion if reached the target
    if data_misfit.chi_factor(model) <= 1.0:
        break

    # Cool down beta by a factor of 2.0
    beta.update()

# Last model is the inverted one
inverted_model = model.copy()
```

Inversion with beta-cooling schedule

1. Define objective function with high value for β:

$$\phi(\mathbf{m}) = \phi_\mathrm{d}(\mathbf{m}) + \beta_0 \phi_\mathrm{m}(\mathbf{m})$$

2. Minimize objective function:

$$\mathbf{m}_{k+1} = \min_{\mathbf{m}} \phi(\mathbf{m})$$

3. Stop the inversion if a certain condition is met.

4. Cool down β by a constant factor:

$$\beta_{k+1} = \frac{\beta_k}{2}$$

```
# Define objective function with dynamic beta
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Run inversion with beta-cooling schedule
model = initial_model.copy()
while True:
    # Perform a minimization step
    model = minimizer(phi, model)

    # Finish the inversion if reached the target
    if data_misfit.chi_factor(model) <= 1.0:
        break

    # Cool down beta by a factor of 2.0
    beta.update()

# Last model is the inverted one
inverted_model = model.copy()
```

Inversion with beta-cooling schedule

1. Define objective function with high value for β:

$$\phi(\mathbf{m}) = \phi_\mathrm{d}(\mathbf{m}) + \beta_0 \phi_\mathrm{m}(\mathbf{m})$$

2. Minimize objective function:

$$\mathbf{m}_{k+1} = \min_{\mathbf{m}} \phi(\mathbf{m})$$

3. Stop the inversion if a certain condition is met.
4. Cool down β by a constant factor:

$$\beta_{k+1} = \frac{\beta_k}{2}$$

```python
# Define objective function with dynamic beta
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Run inversion with beta-cooling schedule
model = initial_model.copy()
while True:
    # Perform a minimization step
    model = minimizer(phi, model)

    # Finish the inversion if reached the target
    if data_misfit.chi_factor(model) <= 1.0:
        break

    # Cool down beta by a factor of 2.0
    beta.update()

# Last model is the inverted one
inverted_model = model.copy()
```

Inversion with beta-cooling schedule

1. Define objective function with high value for β:

$$\phi(\mathbf{m}) = \phi_\mathrm{d}(\mathbf{m}) + \beta_0 \phi_\mathrm{m}(\mathbf{m})$$

2. Minimize objective function:

$$\mathbf{m}_{k+1} = \min_{\mathbf{m}} \phi(\mathbf{m})$$

3. Stop the inversion if a certain condition is met.
4. Cool down β by a constant factor:

$$\beta_{k+1} = \frac{\beta_k}{2}$$

```python
# Define objective function with dynamic beta
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Run inversion with beta-cooling schedule
model = initial_model.copy()
while True:
    # Perform a minimization step
    model = minimizer(phi, model)

    # Finish the inversion if reached the target
    if data_misfit.chi_factor(model) <= 1.0:
        break

    # Cool down beta by a factor of 2.0
    beta.update()

# Last model is the inverted one
inverted_model = model.copy()
```

The `Inversion` class

The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```

The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```

The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```python
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```

Defines a condition for the inversion to stop.

`ChiTarget`:
finish if χ factor ≤ 1

The `Inversion` class

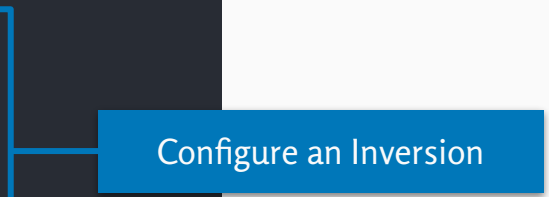
Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```



The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```



The `Inversion` class

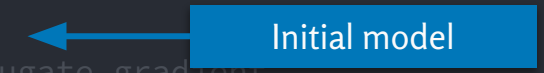
Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```



The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```



The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```

List of directives

The Inversion will execute these callables after each iteration.

E.g.: cool down beta

The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```



The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)

# Run the inversion
inverted_model = inversion.run()
```

The `Inversion` class

Configure a whole inversion in a single `Inversion` object.

```python
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_grad
    directives=[beta.update]
    stopping_criterion=chi_t
)

# Run the inversion
inverted_model = inversion.run()
```

Customizable inversion log

Iteration	β	ϕ_d	ϕ_m	β ϕ_m	ϕ	χ
0	1.00e+07	1.67e+06	0.00e+00	0.00e+00	1.67e+06	1.74e+03
1	1.00e+07	1.40e+05	2.69e-02	2.69e+05	4.08e+05	1.46e+02
2	5.00e+06	6.38e+04	3.74e-02	1.87e+05	2.51e+05	6.64e+01
3	2.50e+06	2.60e+04	4.79e-02	1.20e+05	1.46e+05	2.71e+01
4	1.25e+06	9.80e+03	5.67e-02	7.09e+04	8.07e+04	1.02e+01
5	6.25e+05	3.72e+03	6.34e-02	3.96e+04	4.33e+04	3.87e+00
6	3.12e+05	1.62e+03	6.79e-02	2.12e+04	2.28e+04	1.68e+00
7	1.56e+05	8.68e+02	7.12e-02	1.11e+04	1.20e+04	9.04e-01

🎉 Inversion successfully finished due to stopping criterion.

The `Inversion` class

Alternatively, we can **iterate** over it:

```python
# Define objective function
beta = CooledMultiplier(1e7, cooling_factor=2)
phi = data_misfit + beta * model_norm

# Define a stopping condition
chi_target = ChiTarget(data_misfit)

# Create an Inversion object
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=conjugate_gradient,
    directives=[beta.update],
    stopping_criterion=chi_target,
)
```

```python
# Iterate over the Inversion
for model in inversion:
    # Insert custom code here
    ...

    # Print information
    print(data_misfit.chi_factor(model))

    # Save stuff to disk
    np.save(fname, model)

# Last model is the inverted one
inverted_model = model.copy()
```

Bring your own code!

Define your own `Objective` class

Easily define your own objective function class.

```
from inversion_ideas.base import Objective

class MyRegularization(Objective):
    "Define my custom regularization."

    def __call__(self, model):
        ...

    def gradient(self, model):
        ...

    def hessian(self, model):
        ...
```

Define your own `Objective` class

Easily define your own objective function class.

```python
from inversion_ideas.base import Objective

class MyRegularization(Objective):
    "Define my custom regularization."

    def __call__(self, model):
        ...

    def gradient(self, model):
        ...

    def hessian(self, model):
        ...
```

Just use it to run an inversion with it.

```python
# Define an instance of the new regularization
my_regularization = MyRegularization(...)

# Create an objective function that contains it
phi = data_misfit + beta * my_regularization

# Define an Inversion with this function
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=minimizer,
    ...
)

# Run the inversion
inversion.run()
```

Define your own `Objective` class

Easily define your own objective function class.

```python
from inversion_ideas.base import Objective

class MyRegularization(Objective):
    "Define my custom regularization."

    def __call__(self, model):
        ...

    def gradient(self, model):
        ...

    def hessian(self, model):
        ...
```

Just use it to run an inversion with it.

```python
# Define an instance of the new regularization
my_regularization = MyRegularization(...)

# Create an objective function that contains it
phi = data_misfit + beta * my_regularization

# Define an Inversion with this function
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=minimizer,
    ...
)

# Run the inversion
inversion.run()
```

Define your own `Objective` class

Easily define your own objective function class.

```python
from inversion_ideas.base import Objective

class MyRegularization(Objective):
    "Define my custom regularization."

    def __call__(self, model):
        ...

    def gradient(self, model):
        ...

    def hessian(self, model):
        ...
```

Just use it to run an inversion with it.

```python
# Define an instance of the new regularization
my_regularization = MyRegularization(...)

# Create an objective function that contains it
phi = data_misfit + beta * my_regularization

# Define an Inversion with this function
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=minimizer,
    ...
)

# Run the inversion
inversion.run()
```

Define your own `Objective` class

Easily define your own objective function class.

```python
from inversion_ideas.base import Objective

class MyRegularization(Objective):
    "Define my custom regularization."

    def __call__(self, model):
        ...

    def gradient(self, model):
        ...

    def hessian(self, model):
        ...
```

Just use it to run an inversion with it.

```python
# Define an instance of the new regularization
my_regularization = MyRegularization(...)

# Create an objective function that contains it
phi = data_misfit + beta * my_regularization

# Define an Inversion with this function
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=minimizer,
    ...
)

# Run the inversion
inversion.run()
```

Plug your own `Simulation`

Write your own `Simulation` class and implement its **evaluation** and **derivative**.

```python
from inversion_ideas.base import Simulation

class MySimulation(Simulation):
    "Define our custom simulation."""

    def __call__(self, model):
        ...

    def jacobian(self, model):
        ...

simulation = MySimulation(...)

# Define a data misfit that uses it
data_misfit = ii.DataMisfit(data, uncertainties, simulation)

# Use this data misfit in an inversion
phi = data_misfit + beta * model_norm
inversion = ii.Inversion(phi, ...)
inversion.run()
```

Plug your own `Simulation`

Write your own `Simulation` class and implement its **evaluation** and **derivative**.

```python
from inversion_ideas.base import Simulation

class MySimulation(Simulation):
    "Define our custom simulation."""

    def __call__(self, model):
        ...

    def jacobian(self, model):
        ...

simulation = MySimulation(...)

# Define a data misfit that uses it
data_misfit = ii.DataMisfit(data, uncertainties, simulation)

# Use this data misfit in an inversion
phi = data_misfit + beta * model_norm
inversion = ii.Inversion(phi, ...)
inversion.run()
```

Plug your own `Simulation`

Write your own `Simulation` class and implement its **evaluation** and **derivative**.

```python
from inversion_ideas.base import Simulation

class MySimulation(Simulation):
    "Define our custom simulation."""

    def __call__(self, model):
        ...

    def jacobian(self, model):
        ...

simulation = MySimulation(...)

# Define a data misfit that uses it
data_misfit = ii.DataMisfit(data, uncertainties, simulation)

# Use this data misfit in an inversion
phi = data_misfit + beta * model_norm
inversion = ii.Inversion(phi, ...)
inversion.run()
```

Plug your own `Simulation`

Write your own `Simulation` class and implement its **evaluation** and **derivative**.

```python
from inversion_ideas.base import Simulation

class MySimulation(Simulation):
    "Define our custom simulation."""

    def __call__(self, model):
        ...

    def jacobian(self, model):
        ...

simulation = MySimulation(...)

# Define a data misfit that uses it
data_misfit = ii.DataMisfit(data, uncertainties, simulation)

# Use this data misfit in an inversion
phi = data_misfit + beta * model_norm
inversion = ii.Inversion(phi, ...)
inversion.run()
```

Plug your own `Simulation`

Write your own `Simulation` class and implement its **evaluation** and **derivative**.

```python
from inversion_ideas.base import Simulation

class MySimulation(Simulation):
    "Define our custom simulation."""

    def __call__(self, model):
        ...

    def jacobian(self, model):
        ...

simulation = MySimulation(...)

# Define a data misfit that uses it
data_misfit = ii.DataMisfit(data, uncertainties, simulation)

# Use this data misfit in an inversion
phi = data_misfit + beta * model_norm
inversion = ii.Inversion(phi, ...)
inversion.run()
```

Use any minimizer!

The framework is compatible with SciPy minimizers

```python
from scipy.optimize import minimize

# Define our objective function
phi = data_misfit + beta * model_norm

# Use SciPy minimizer
inverted_model = minimize(
    phi,
    initial_model,
    method="BFGS",
    jac=phi.gradient,
)
```

Use any minimizer!

The framework is compatible with SciPy minimizers

```python
from scipy.optimize import minimize

# Define our objective function
phi = data_misfit + beta * model_norm

# Use SciPy minimizer
inverted_model = minimize(
    phi,
    initial_model,
    method="BFGS",
    jac=phi.gradient,
)
```

Or you can write your own

```python
def custom_minimizer(objective_fun, initial_model):
    """Define a custom minimizer."""
    ...
    return inverted_model

# Use custom minimizer in inversion
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=custom_minimizer,
    stopping_criterion=condition,
    directives=[...],
)

# Run inversion
inversion.run()
```

Use any minimizer!

The framework is compatible with SciPy minimizers

```python
from scipy.optimize import minimize

# Define our objective function
phi = data_misfit + beta * model_norm

# Use SciPy minimizer
inverted_model = minimize(
    phi,
    initial_model,
    method="BFGS",
    jac=phi.gradient,
)
```

Or you can write your own

```python
def custom_minimizer(objective_fun, initial_model):
    """Define a custom minimizer."""
    ...
    return inverted_model

# Use custom minimizer in inversion
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=custom_minimizer,
    stopping_criterion=condition,
    directives=[...],
)

# Run inversion
inversion.run()
```

Use any minimizer!

The framework is compatible with SciPy minimizers

```python
from scipy.optimize import minimize

# Define our objective function
phi = data_misfit + beta * model_norm

# Use SciPy minimizer
inverted_model = minimize(
    phi,
    initial_model,
    method="BFGS",
    jac=phi.gradient,
)
```

Or you can write your own

```python
def custom_minimizer(objective_fun, initial_model):
    """Define a custom minimizer."""
    ...
    return inverted_model

# Use custom minimizer in inversion
inversion = ii.Inversion(
    phi,
    initial_model,
    minimizer=custom_minimizer,
    stopping_criterion=condition,
    directives=[...],
)

# Run inversion
inversion.run()
```

Aid research through modularity

Inversion with segmentations

Geophysical inversion in primal-dual space:

- Plug ML image segmentation into the inversion
- Better define geophysical structures
- Perform minimizations in primal (model) and dual spaces with two linked objective functions
- Use of ADMM minimizer for the dual space



John Kuttai

```python
# Model space
segmentation_norm = ii.SegmentationNorm(...)
model_norm = (
    alpha_s * smallness + alpha_x * flatness_x
    + alpha_y * flatness_y + 1e-6 * segmentation_norm
)
phi = data_misfit + model_norm

# Dual space
phi_dual_space = ii.SegmentationTotalVariation(...)

# Minimizers
minimizer_model = ii.GaussNewtonConjugateGradient(...)
minimizer_dual = ii.PrimalDual(...)

for i in range(n_iters):
    # Minimize in the model space
    *_, model = minimizer_model(phi, model)

    # Perform a segmentation classification
    cl, Z = mix_model(model, Z)

    # Pass classification result to dual minimizer
    minimizer_dual.cl = torch.tensor(cl, ...)

    # Minimize in the dual space
    Z = minimizer_dual(phi_seg_obj, model, Z=Z)

    # Update quantities connecting the two spaces
    segmentation_norm.Z, segmentation.c = Z, cl
    mix_model.cl = cl
    smallness.reference_model = Z @ cl
```

Inversion with segmentations

Geophysical inversion in primal-dual space:

- Plug ML image segmentation into the inversion
- Better define geophysical structures
- Perform minimizations in primal (model) and dual spaces with two linked objective functions
- Use of ADMM minimizer for the dual space



John Kuttai

```
# Model space
segmentation_norm = ii.SegmentationNorm(...)
model_norm = (
    alpha_s * smallness + alpha_x * flatness_x
    + alpha_y * flatness_y + 1e-6 * segmentation_norm
)
phi = data_misfit + model_norm

# Dual space
phi_dual_space = ii.SegmentationTotalVariation(...)

# Minimizers
minimizer_model = ii.GaussNewtonConjugateGradient(...)
minimizer_dual = ii.PrimalDual(...)

for i in range(n_iters):
    # Minimize in the model space
    *_, model = minimizer_model(phi, model)

    # Perform a segmentation classification
    cl, Z = mix_model(model, Z)

    # Pass classification result to dual minimizer
    minimizer_dual.cl = torch.tensor(cl, ...)

    # Minimize in the dual space
    Z = minimizer_dual(phi_seg_obj, model, Z=Z)

    # Update quantities connecting the two spaces
    segmentation_norm.Z, segmentation.c = Z, cl
    mix_model.cl = cl
    smallness.reference_model = Z @ cl
```

Inversion with segmentations

Geophysical inversion in primal-dual space:

- Plug ML image segmentation into the inversion
- Better define geophysical structures
- Perform minimizations in primal (model) and dual spaces with two linked objective functions
- Use of ADMM minimizer for the dual space



John Kuttai

```
# Model space
segmentation_norm = ii.SegmentationNorm(...)
model_norm = (
    alpha_s * smallness + alpha_x * flatness_x
    + alpha_y * flatness_y + 1e-6 * segmentation_norm
)
phi = data_misfit + model_norm

# Dual space
phi_dual_space = ii.SegmentationTotalVariation(...)

# Minimizers
minimizer_model = ii.GaussNewtonConjugateGradient(...)
minimizer_dual = ii.PrimalDual(...)

for i in range(n_iters):
    # Minimize in the model space
    *_, model = minimizer_model(phi, model)

    # Perform a segmentation classification
    cl, Z = mix_model(model, Z)

    # Pass classification result to dual minimizer
    minimizer_dual.cl = torch.tensor(cl, ...)

    # Minimize in the dual space
    Z = minimizer_dual(phi_seg_obj, model, Z=Z)

    # Update quantities connecting the two spaces
    segmentation_norm.Z, segmentation.c = Z, cl
    mix_model.cl = cl
    smallness.reference_model = Z @ cl
```

Inversion with segmentations

Geophysical inversion in primal-dual space:

- Plug ML image segmentation into the inversion
- Better define geophysical structures
- Perform minimizations in primal (model) and dual spaces with two linked objective functions
- Use of ADMM minimizer for the dual space



John Kuttai

```
# Model space
segmentation_norm = ii.SegmentationNorm(...)
model_norm = (
    alpha_s * smallness + alpha_x * flatness_x
    + alpha_y * flatness_y + 1e-6 * segmentation_norm
)
phi = data_misfit + model_norm

# Dual space
phi_dual_space = ii.SegmentationTotalVariation(...)

# Minimizers
minimizer_model = ii.GaussNewtonConjugateGradient(...)
minimizer_dual = ii.PrimalDual(...)

for i in range(n_iters):
    # Minimize in the model space
    *_, model = minimizer_model(phi, model)

    # Perform a segmentation classification
    cl, Z = mix_model(model, Z)

    # Pass classification result to dual minimizer
    minimizer_dual.cl = torch.tensor(cl, ...)

    # Minimize in the dual space
    Z = minimizer_dual(phi_seg_obj, model, Z=Z)

    # Update quantities connecting the two spaces
    segmentation_norm.Z, segmentation.c = Z, cl
    mix_model.cl = cl
    smallness.reference_model = Z @ cl
```

Inversion with segmentations

Geophysical inversion in primal-dual space:

- Plug ML image segmentation into the inversion
- Better define geophysical structures
- Perform minimizations in primal (model) and dual spaces with two linked objective functions
- Use of ADMM minimizer for the dual space



John Kuttai

```
# Model space
segmentation_norm = ii.SegmentationNorm(...)
model_norm = (
    alpha_s * smallness + alpha_x * flatness_x
    + alpha_y * flatness_y + 1e-6 * segmentation_norm
)
phi = data_misfit + model_norm

# Dual space
phi_dual_space = ii.SegmentationTotalVariation(...)

# Minimizers
minimizer_model = ii.GaussNewtonConjugateGradient(...)
minimizer_dual = ii.PrimalDual(...)

for i in range(n_iters):
    # Minimize in the model space
    *_, model = minimizer_model(phi, model)

    # Perform a segmentation classification
    cl, Z = mix_model(model, Z)

    # Pass classification result to dual minimizer
    minimizer_dual.cl = torch.tensor(cl, ...)

    # Minimize in the dual space
    Z = minimizer_dual(phi_seg_obj, model, Z=Z)

    # Update quantities connecting the two spaces
    segmentation_norm.Z, segmentation.c = Z, cl
    mix_model.cl = cl
    smallness.reference_model = Z @ cl
```

Plugging an empymod simulation

```python
import empymod, empygrad
from inversion_ideas.base import Simulation

class VTISimulation(Simulation):

    def __call__(self, model):
        res, aniso = self.unpack(model)
        d = empymod.bipole(**self.survey, res=res, aniso=aniso)
        return stack_real_imag(d)

    def jacobian(self, model):
        res, aniso = self.unpack(model)
        _, J = empygrad.bipole(**self.survey, res=res, aniso=aniso,
                               jac=["res", "aniso"])
        J = np.hstack([J["res"], J["aniso"]])
        return stack_real_imag(J)
```

emπmod



Wouter Deleersnyder

Stay for the next talk to learn more about it!

Project status

Project status

⚠️ Still in experimental stages

- Still prototyping the framework.
- Experimenting with design.
- Expect breaking changes.
- But usable! Feel free to try it out!

Documentation

simpeg.xyz/inversion-design-ideas



User Guide

API Reference

inversion-ideas documentation | User Guide | API Reference | Search Ctrl + K

Collapse Sidebar

Section Navigation

Getting started
- What is this new framework?
- Installing

Fundamentals
- Inversion problem
- Objective function
- Minimizer
  - Preconditioners
- Inversions
  - Beta-cooling schedule
  - The `Inversion` class
- Conditions
- Hyperparameters
- Simulations
  - Plugging in SimPEG simulations

Examples
- Gravity Inversion

How To
- Logging and debugging
- Extend the framework

User Guide > Objective function

Objective function

Defining objective functions

Any objective function is defined as a scalar field $\phi : \mathbb{R}^M \rightarrow \mathbb{R}$, i.e. it takes a model vector $\mathbf{m}$ with M elements and returns a single real value.

In the new SimPEG's inversion framework, objective functions are represented by a child of the `inversion_ideas.base.Objective`, like the `inversion_ideas.DataMisfit` for example.

```
import numpy as np
import inversion_ideas as ii

# Define a linear regressor as a simulation
n_data, n_params = 3, 5
simulation = ii.simulations.LinearRegressor.create_random(n_data, n_params, seed=42)

# Create some synthetic data and uncertainties
true_model = np.array([-5., 2., -3., 4., 1.])
std = 1e-2
data = simulation(true_model) + np.random.default_rng(seed=42).normal(size=n_data, scale=std)
uncertainties = std * np.ones_like(data)

# Define a data misfit function
data_misfit = ii.DataMisfit(data, uncertainties, simulation)
data_misfit
```

$$\phi_d(\mathbf{m})$$

Once defined, the objective functions can only be evaluated on model vectors with `n_params` elements. We can

Objective functions — inv

inversion-ideas documentation | User Guide | API Reference | Search Ctrl + K

Collapse Sidebar

Section Navigation
- Objective functions
  - inversion_ideas.DataMisfit
  - inversion_ideas.SimpleSmallness
  - inversion_ideas.SimpleFlatness
  - inversion_ideas.Smallness
  - inversion_ideas.Flatness
  - inversion_ideas.SparseSmallness
- Minimizers
- Preconditioners
- Directives
- Conditions
- Simulations
- Inversions and Recipes
- Custom linear operators
- Utilities
- Hyperparameters
- Decorators
- Errors and warnings
- Base submodule

API Reference > Objective functions

Objective functions

Misfits

`DataMisfit` (data, uncertainty, simulation, *)	L2 data misfit.

Regularizations

General purpose:

`SimpleSmallness` (n_params[, weights, ...])	Simple smallness regularization.
`SimpleFlatness` (n_params)	Simple flatness regularization.

Mesh-based:

`Smallness` (mesh, *[, active_cells, ...])	Smallness regularization.
`Flatness` (mesh, direction, *[, active_cells, ...])	Flatness regularization.
`SparseSmallness` (mesh, *, norm[, ...])	Smallness regularization using lp norm.

Previous: API Reference

Next: inversion_ideas.DataMisfit

Testing 🧪

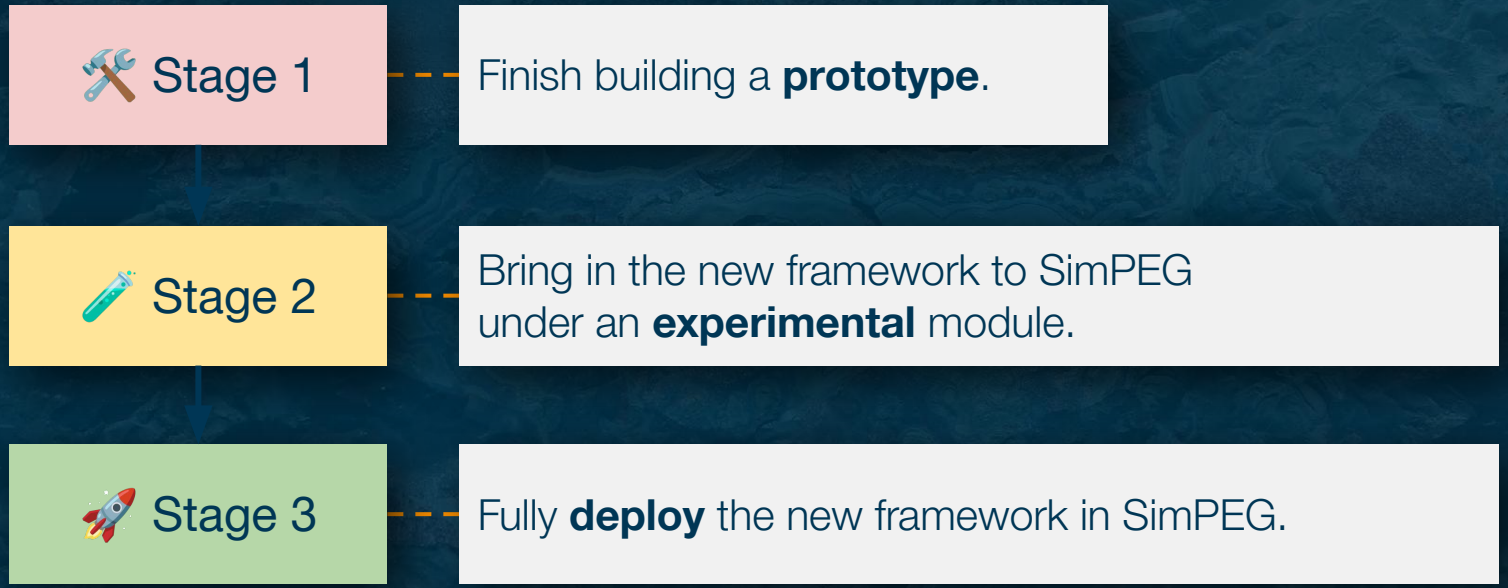
Growing test suite

```
======================================== tests coverage ========================================
_____________________ coverage: platform linux, python 3.14.3-final-0 _____________________

Name                                                                                 Stmts   Miss Branch BrPart  Cover
-----------------------------------------------------------------------------------------------------------------
src/inversion_ideas/_utils.py                                                           40     14     12      2    58%
src/inversion_ideas/base/_utils.py                                                      20      0      8      0   100%
src/inversion_ideas/base/conditions.py                                                  84      0     16      0   100%
src/inversion_ideas/base/directive.py                                                    6      1      0      0    83%
src/inversion_ideas/base/hyperparams.py                                                200      0     22      0   100%
src/inversion_ideas/base/minimizer.py                                                   20      7      0      0    65%
src/inversion_ideas/base/objective_function.py                                         318      0     96      0   100%
src/inversion_ideas/base/simulation.py                                                  16      0      0      0   100%
...
-----------------------------------------------------------------------------------------------------------------
TOTAL                                                                                 4634    599    722     35    84%
==================================== 493 passed in 6.12s =====================================
```

Future roadmap

Future roadmap



Conclusions



- Developed a **prototype** for a new **inversion framework** for SimPEG.
  - Open-source
  - Python
- Increased **modularity** and **extensibility**.
- **Aid research:** make it **easier to experiment** with new methods.
- **Aid learning:** easier to read and study.



Docs:
`simpeg.xyz/inversion-design-ideas`



Source code:
`simpeg/inversion-design-ideas`

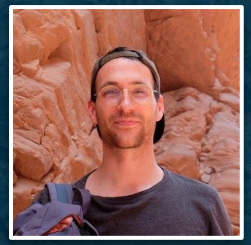
Acknowledgements



Joe Capriotti



Lindsey Heagy



Dom Fournier



John Kuttai



Kathi Maetschke



John Weis



Parth Pokar



Wouter Deleersnyder

Thank you!

Docs:
`simpeg.xyz/inversion-design-ideas`

Source code:
`simpeg/inversion-design-ideas`